

RNN (Recurrent Neural Network)

Traditional ML models
like BoW, Word2Vec

Language translation (Hindi $\rightarrow$ English)

$\rightarrow$ sequence of any given word in

a sentence matters

alot

$\rightarrow$ Are these two sentences

same?

No

ML, Traditional NN

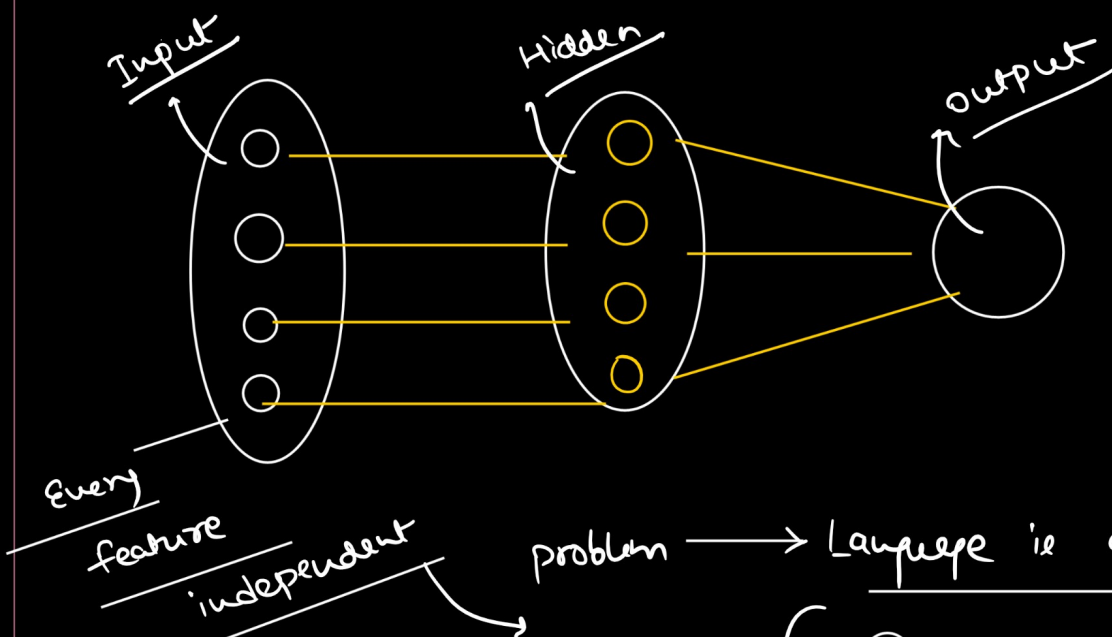
$\rightarrow$ This introduces the concept of

Input features sequence

Traditional Neural Network

are independent

I love Data Science.



RNN

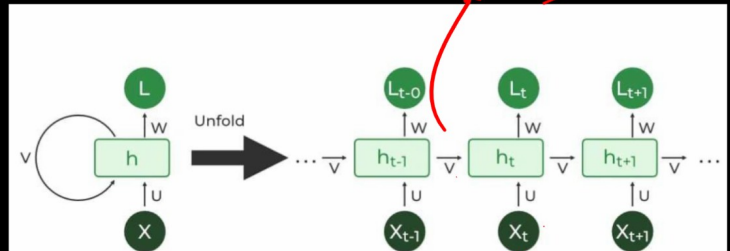
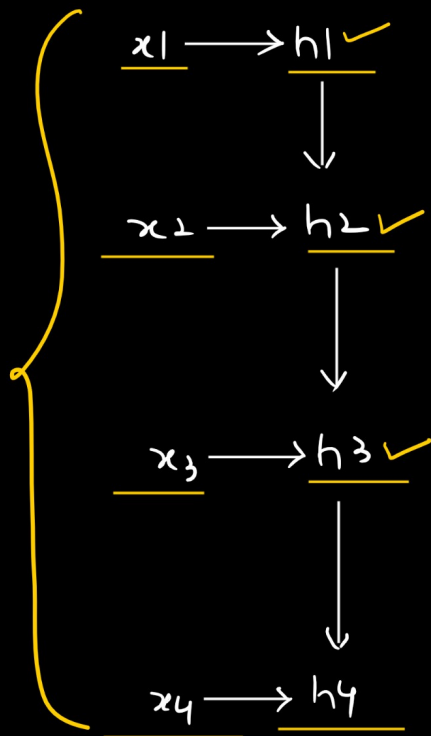
① Word order matters.

② Content matters

③ Dependency matters

Instead of processing words independently,
information flows through time

Architecture



$$h_t = f(\underbrace{w x_t}_{\text{current word}} + \underbrace{u h_{t-1}}_{\text{previous state content}})$$

hidden state

RNN Architecture

I love deep learning.

Language modeling

→ to predict
the next
word token

I love machine

$\underbrace{\quad}_{x_1} \quad \underbrace{\quad}_{x_2} \quad \underbrace{\quad}_{x_3}$

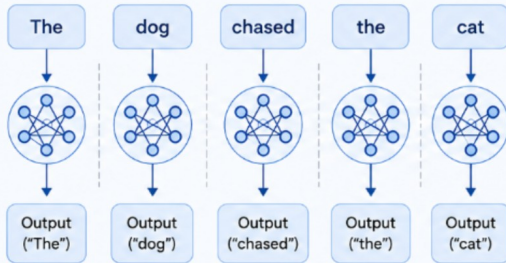
target word
 y

$P(y | x_1, x_2, x_3)$

Why Feedforward Networks Struggle with Sequential Data

Feedforward Neural Network (Processes Independently)

Each word is processed separately.
No memory of previous or future words.

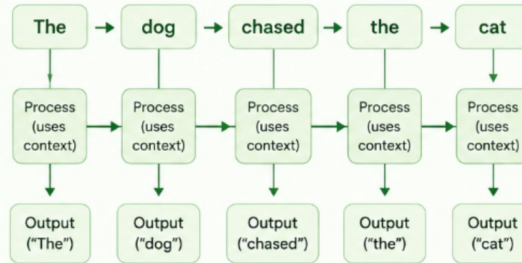


❌ **Limitation:** Cannot capture order, context, or relationships between words.

🧠 Feedforward networks treat every input the same, making them ineffective for sequential data.

Sequential Data (Requires Order & Context)

Each word depends on previous words.
Information flows from one step to the next.



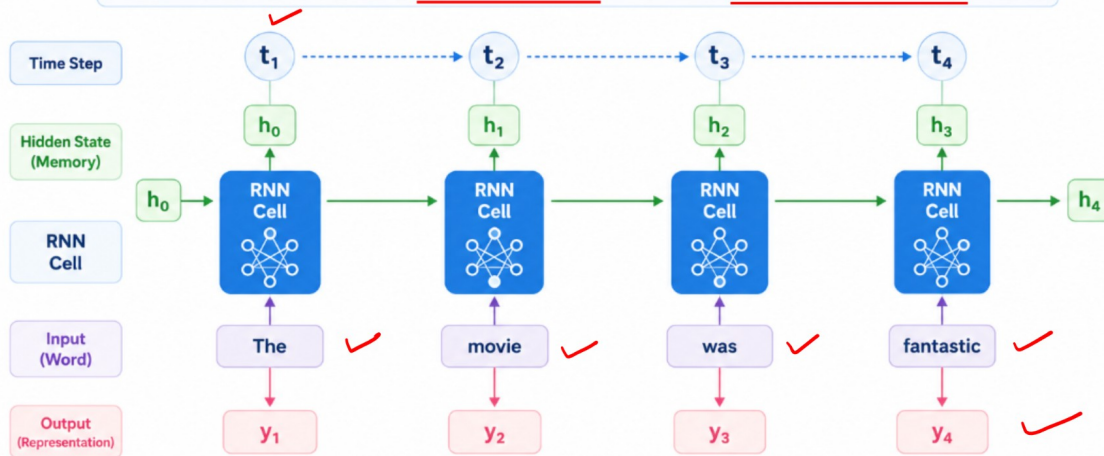
✅ **Advantage:** Captures order, context, and relationships between words.

💡 Sequential models (like RNNs, LSTMs, Transformers) are designed to handle order and context.

★ **Key Insight:** Feedforward networks lack memory — they see words, but not their order.

How an RNN Processes a Sentence.

An RNN reads a sentence one word at a time and carries its memory (hidden state) forward.



- ➡ **Memory Flow:** Hidden state (h) carries information from previous words to the next.
- ➡ **Input:** Current word at each time step.
- ➡ **Output:** Representation or prediction at each step.

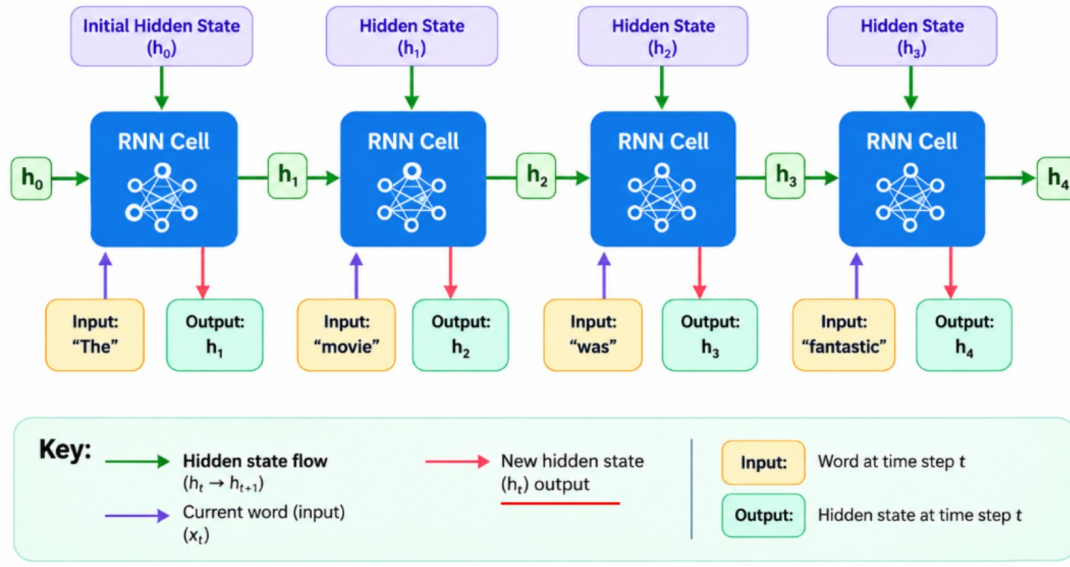


Key Idea

The hidden state acts like memory, so the RNN knows what came before and can understand the context of each word.

RNN Hidden State

An RNN uses its **hidden state** to carry information from one time step to the next.



LSTM $\rightarrow$ Long Short Term Memory

Long-term memory $\Rightarrow$ c-t (cell state)
 $\searrow$ Information highway

Short-term memory $\Rightarrow$ h_t (hidden state)

① forget Gate $\rightarrow$ What should I forget?

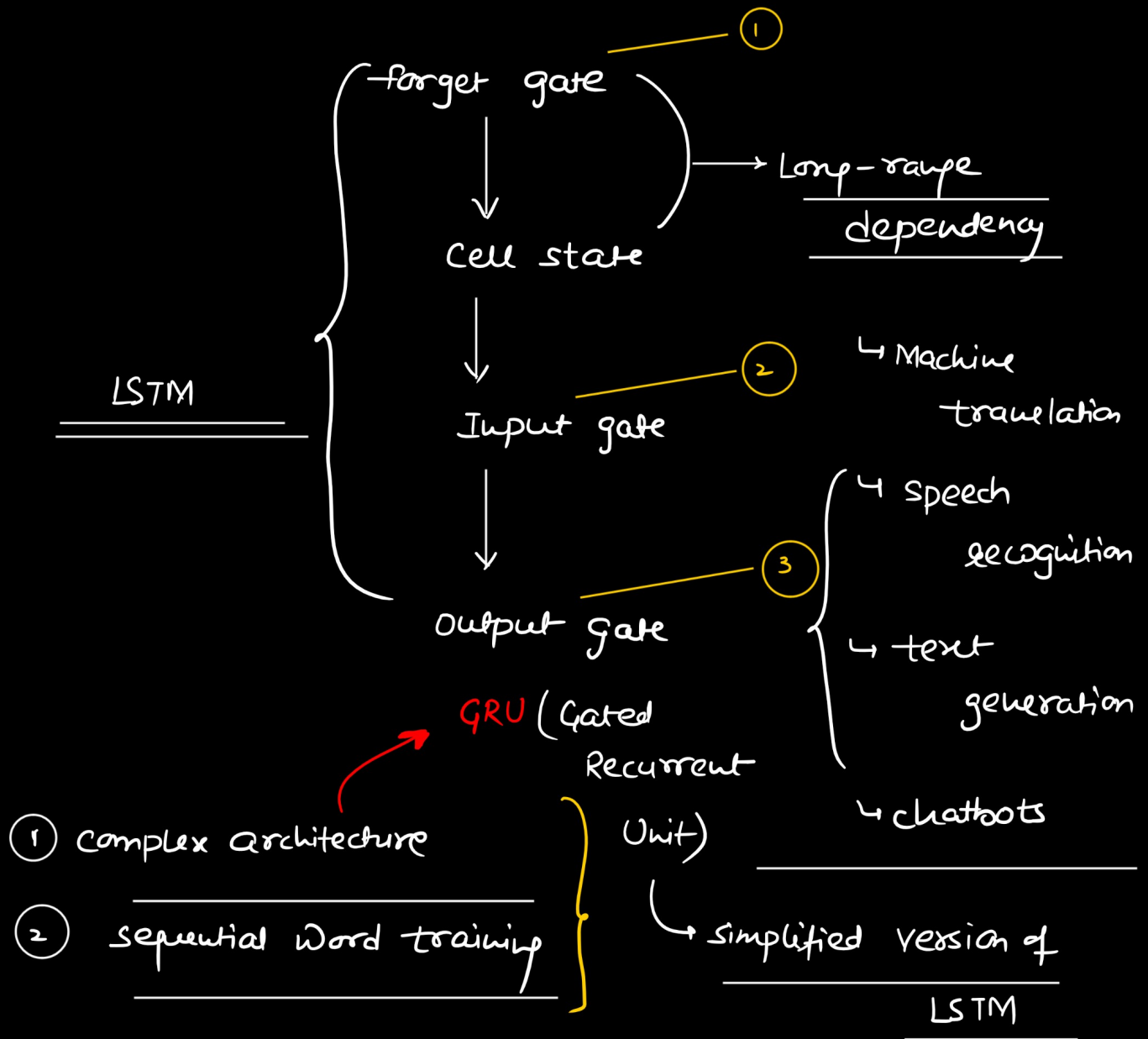
My old phone was broken.

I bought a new phone.

Old phone details can be discarded and
 only new phone details will be
 taken care of by forget gate.

② Input Gate :- What should I remember?
(New information gets stored)

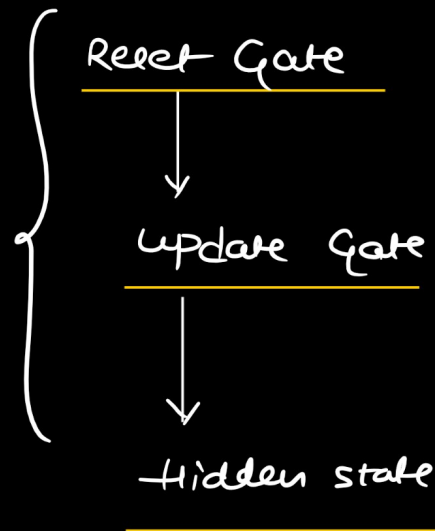
③ Output Gate :- What should I reveal?
(produce prediction)



GRU

① update Gate → forget + Input

② Reset Gate → control the previous memory usage



① fewer parameters

② faster training

③ less memory usage

fast
RNN → No concept of gates

slow ← LSTM → 3 gates (too much complexity)

GRU → 2 gates

↘ faster

Understanding Perplexity

Perplexity measures how well a language model predicts a sequence.

